

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LAB ASSIGNMENT-02

NAME : SALVA SOUMYA

REGISTER NUMBER : 23MID0061

COURSE TITLE : ADVANCED PYTHON PROGRAMMING

COURSE CODE : CSI3007

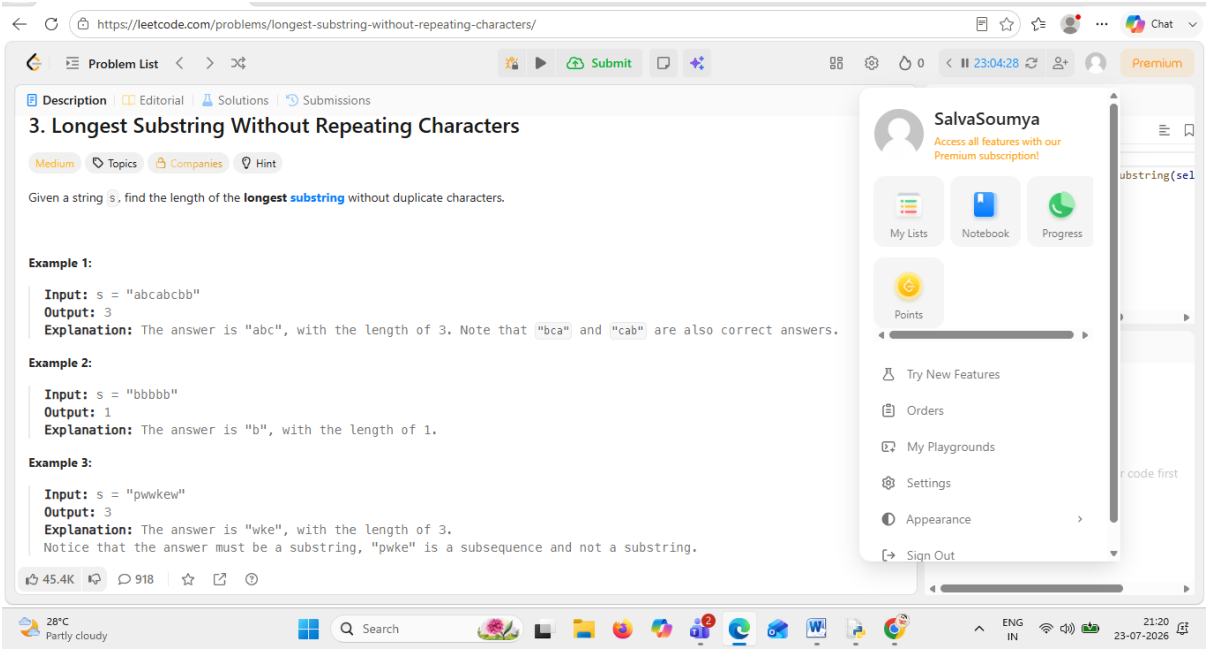
COURSE FACULTY : SHARMILA BANU K

LAB SLOT : L31+L32+L49+L50

SUBMISSION DATE : 23-07-2026

LEETCODE QUESTIONS SOLVING

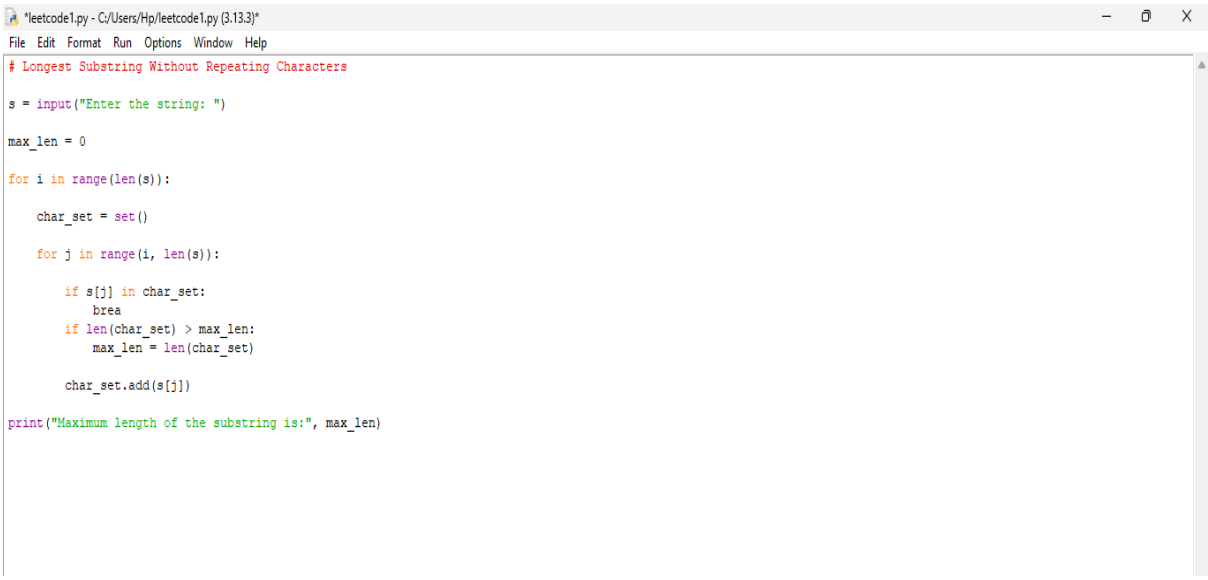
QUESTION -1 :



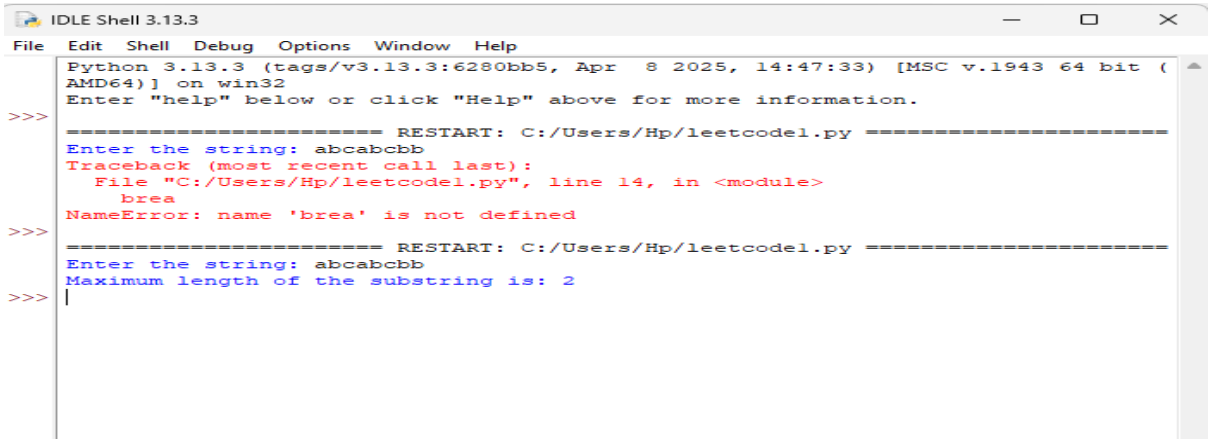
INPUT : A string

OUTPUT : Maximum length of the substring without repeating characters

CODE :



ERROR ENCOUNTED :



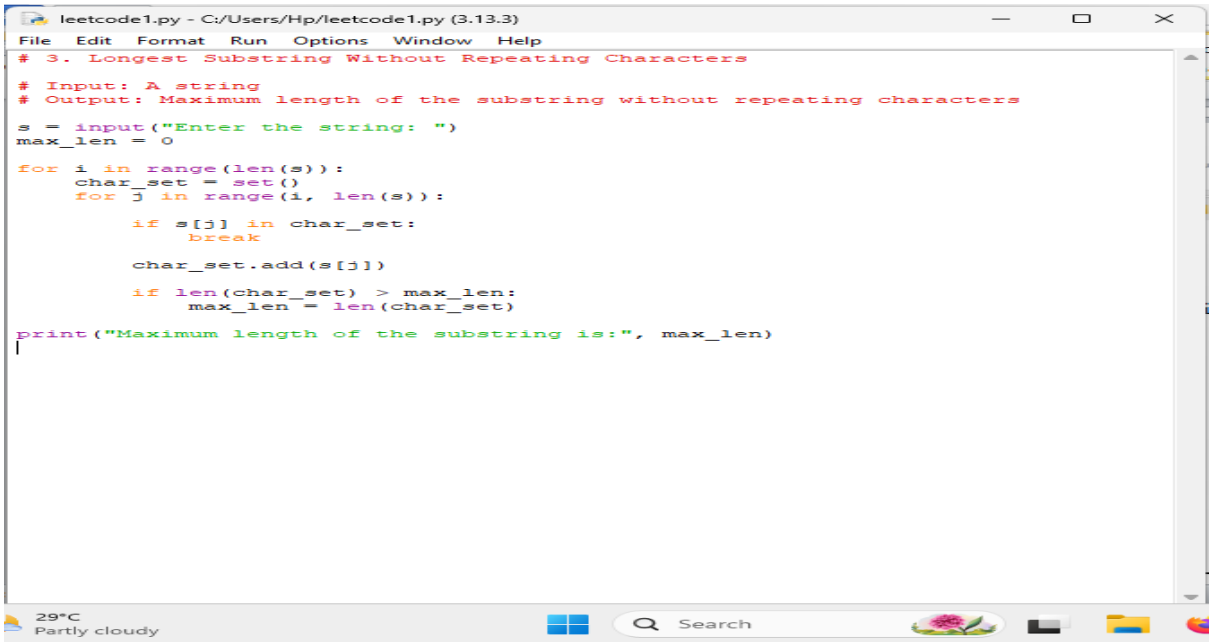
Expected output :

Maximum length of the substring is: 3

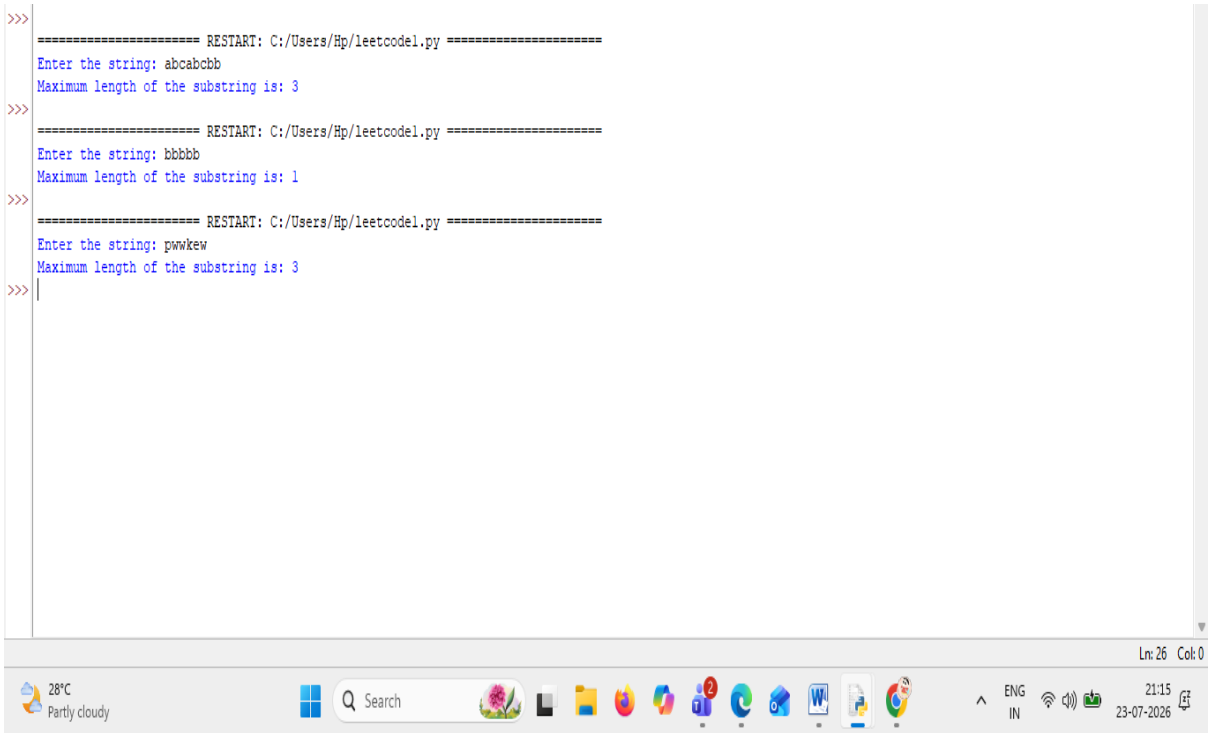
Reason for the error in the output is :

I updated max_len before adding the current character to the set.

CORRECTED CODE :



OUTPUT :



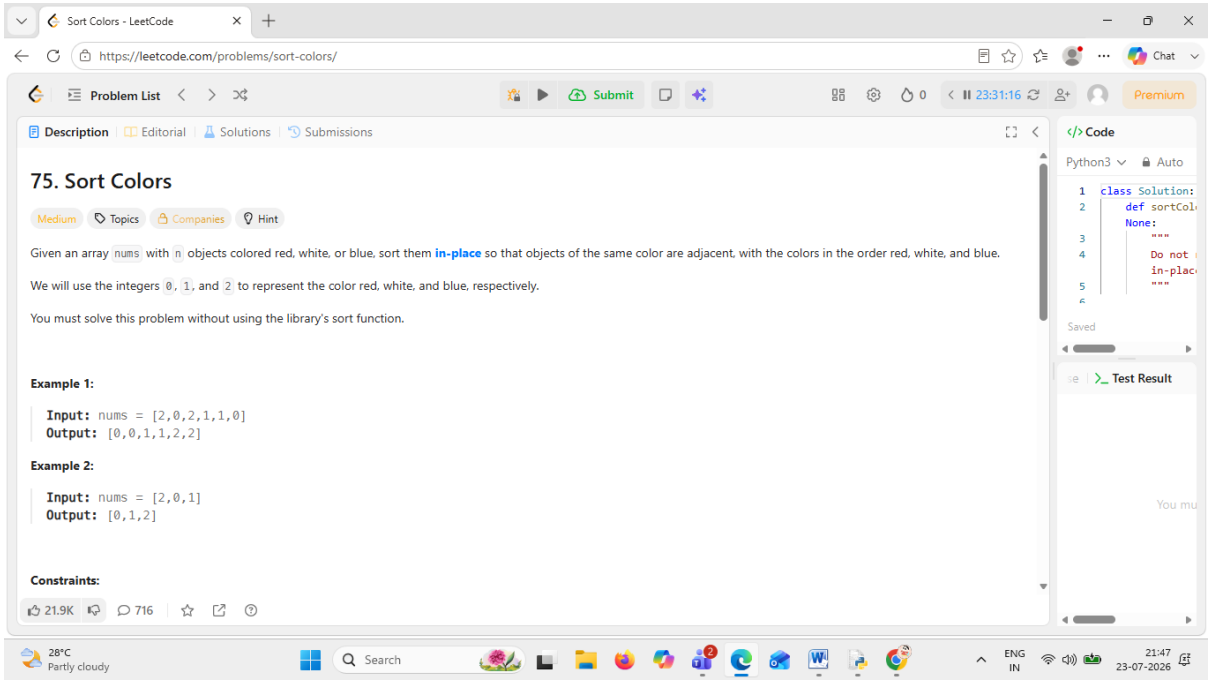
Justification for Data Structure Used :

A **set** is used because it stores only unique elements. It helps to quickly check whether a character is already present in the current substring. It easy to detect duplicate characters and stop checking that substring when a repetition occurs.

Logic Used :

1. Read the input string from the user.
2. Start checking every possible substring by using two loops.
3. Use a **set** to store the unique characters of the current substring.
4. If a duplicate character is found, stop checking that substring using " break ".
5. Compare the current substring length with the maximum length and update it if it is greater.
6. Finally, display the maximum length.

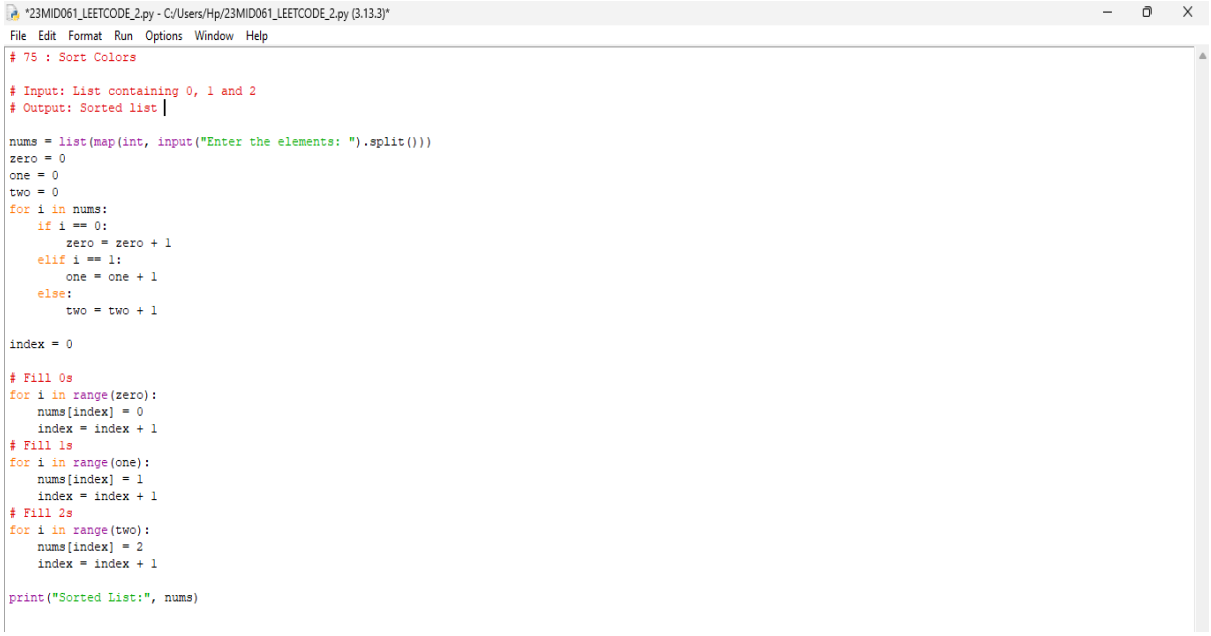
QUESTION -2 :



INPUT : A list of integers containing only 0, 1, and 2.

OUTPUT : The sorted list with all 0s first, followed by 1s, and then 2s.

CODE :




```
# 75 : Sort Colors

# Input: List containing 0, 1 and 2
# Output: Sorted list

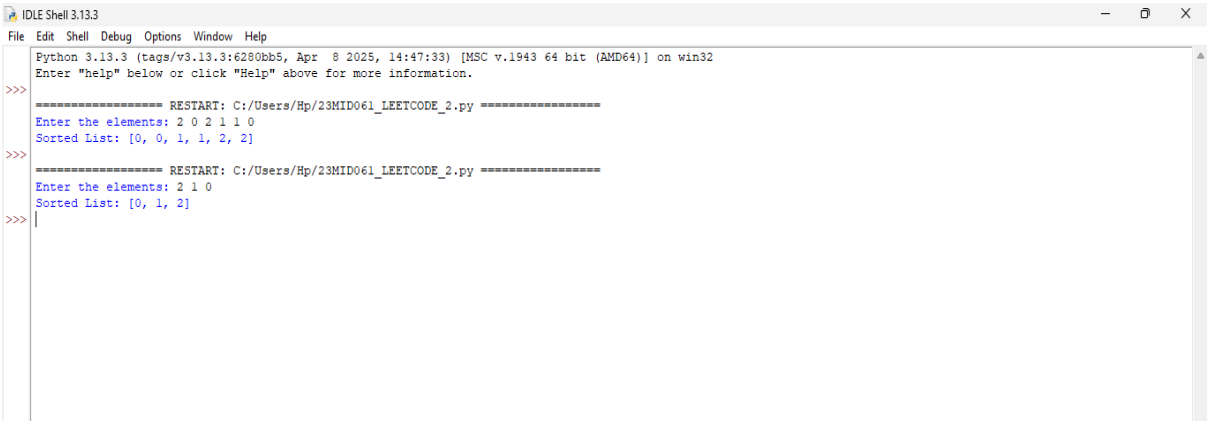
nums = list(map(int, input("Enter the elements: ").split()))
zero = 0
one = 0
two = 0
for i in nums:
    if i == 0:
        zero = zero + 1
    elif i == 1:
        one = one + 1
    else:
        two = two + 1

index = 0

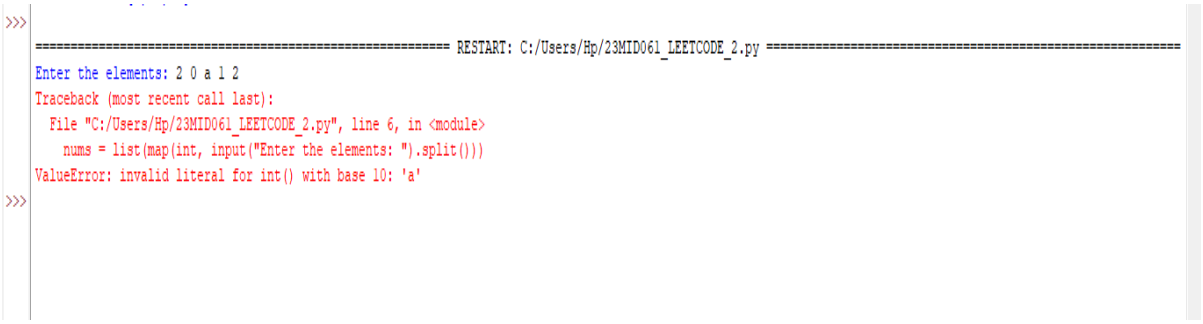
# Fill 0s
for i in range(zero):
    nums[index] = 0
    index = index + 1
# Fill 1s
for i in range(one):
    nums[index] = 1
    index = index + 1
# Fill 2s
for i in range(two):
    nums[index] = 2
    index = index + 1

print("Sorted List:", nums)
```

OUTPUT :



ERROR ENCOUNTERED :



Reason for the error in the output is :

nums = list(map(int, input("Enter the elements: ").split())) .According to the problem , the list nums should contain only the integers 0, 1, and 2**.** Entering a character such as 'a' cannot be converted to an integer, resulting in a ValueError**.**

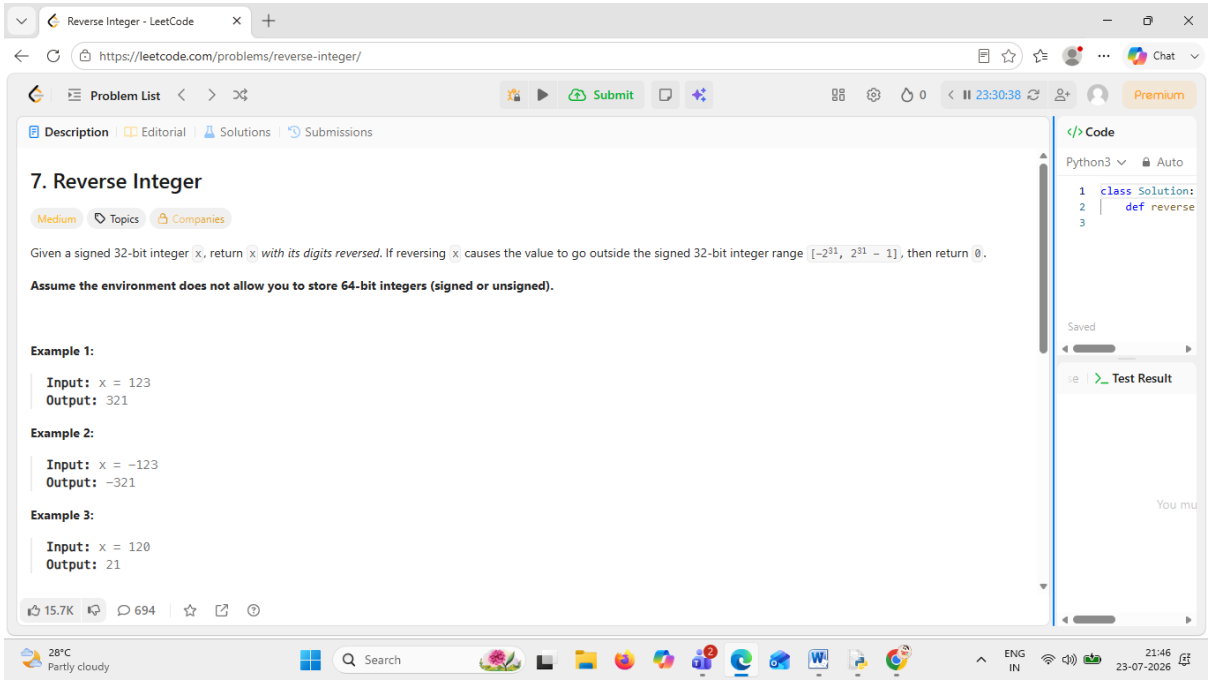
Justification for Data Structure Used :

A **list** is used because it stores multiple integer values and allows easy traversal using loops. Since the input is an array, a list is the most suitable data structure for accessing and arranging the elements.

Logic Used :

1. Read the list of numbers from the user.
2. Count the number of 0s, 1s, and 2s in the list.
3. Replace the elements of the list by first filling all 0s, then 1s, and finally 2s.
4. Display the sorted list.

QUESTION -3 :



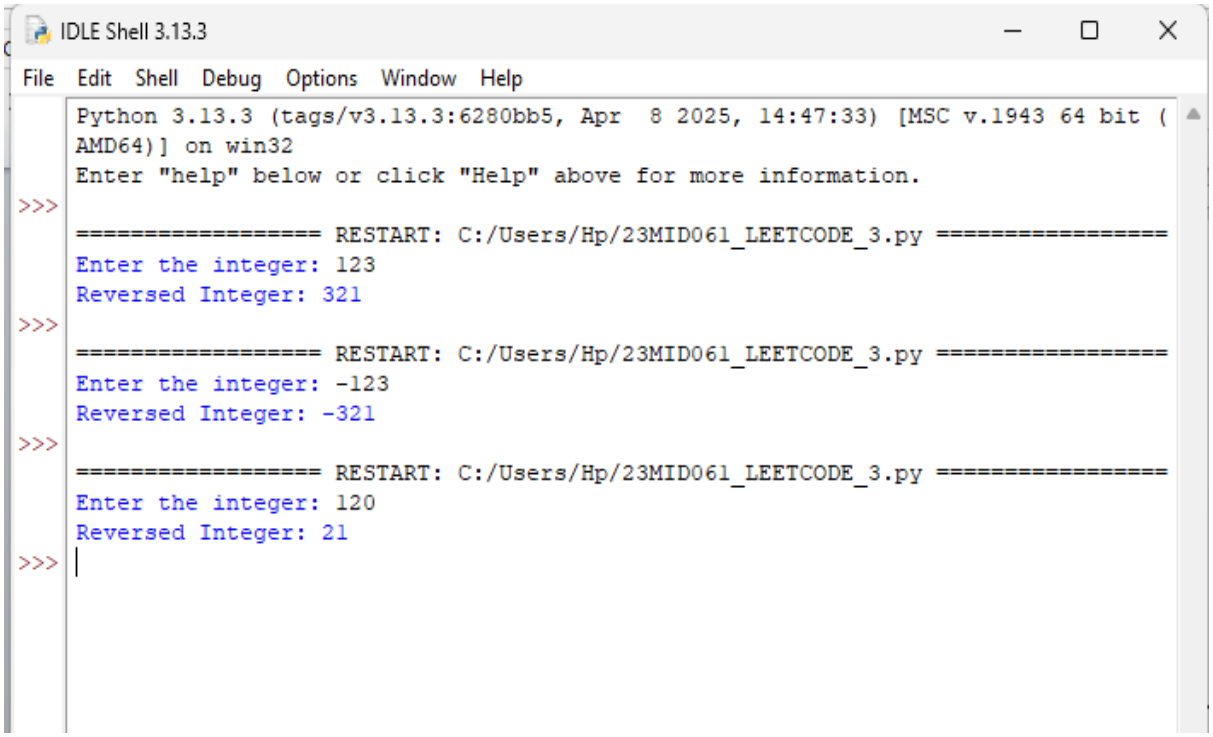
INPUT : A signed integer `x`.

OUTPUT : The reversed integer. If the reversed integer exceeds the 32-bit signed integer range (`-2³¹` to `2³¹ - 1`), output `0`.

CODE :



OUTPUT :



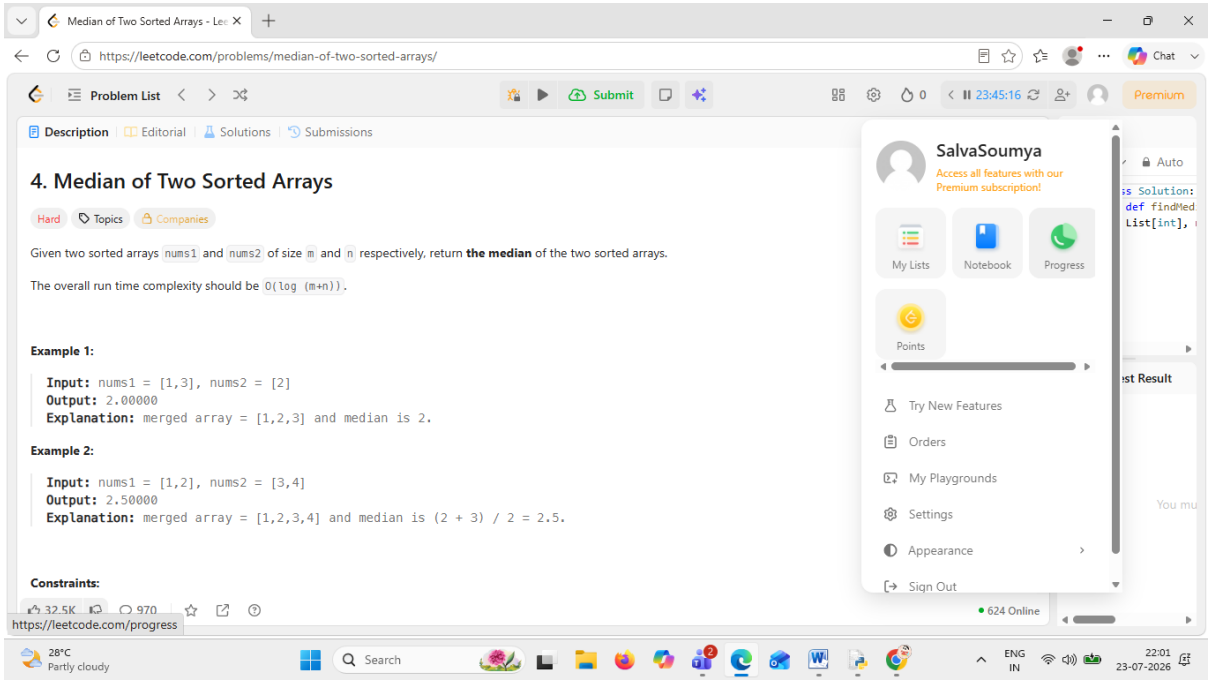
Justification for Data Structure Used :

A **string** is used to easily reverse the digits of the integer using slicing (`[::-1]`). It simplifies the reversing process without using loops or any other arithmetic operations.

Logic Used :

1. Read the integer from the user.
2. Check whether the number is negative and store its sign.
3. Convert the integer to a string and reverse it using string slicing ([::-1]).
4. Convert the reversed string back to an integer, restore the sign, and check whether it lies within the 32-bit signed integer range.
5. If it exceeds the limit, print 0; otherwise, print the reversed integer.

QUESTION -4 :



INPUT : Two sorted integer arrays.

OUTPUT : The median of the merged sorted array.

CODE :

23MID061_LEETCODE_4.py - C:/Users/Hp/23MID061_LEETCODE_4.py (3.13.3)

File Edit Format Run Options Window Help

```
# Median of Two Sorted Arrays

# Input: Two sorted arrays
# Output: Median of the merged array

nums1 = list(map(int, input("Enter first sorted array: ").split()))
nums2 = list(map(int, input("Enter second sorted array: ").split()))

merged = nums1 + nums2
merged.sort()
n = len(merged)

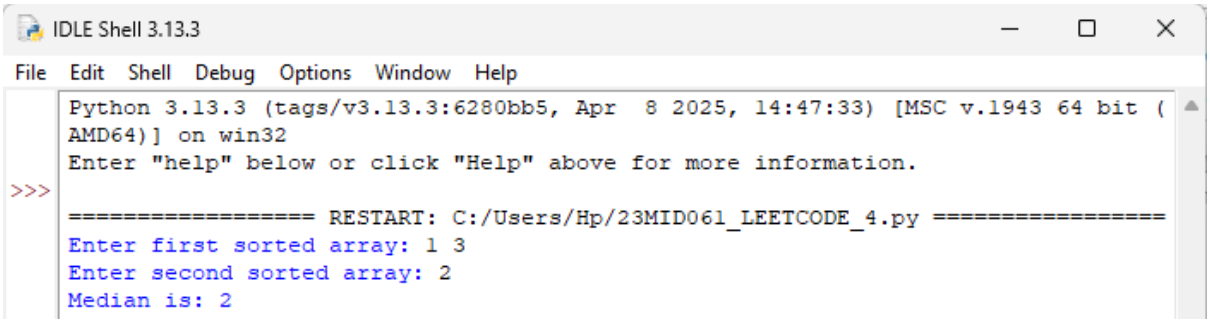
if n % 2 == 1:
    median = merged[n // 2]
else:
    median = (merged[n // 2 - 1] + merged[n // 2]) / 2

print("Median is:", median)
```

Expected Output

Median is: 2.00000

ERROR ENCOUNTED :



Reason for the error in the output is :

The median was stored as an **integer** when the total number of elements was odd. According to the problem, the output should be displayed as a floating-point value with formatting .Therefore, the integer result needed to be converted to a float before displaying the output.

CORRECTED CODE :



OUTPUT :

```
>>>
================== RESTART: C:/Users/Hp/23MID061_LEETCODE_4.py ==================
Enter first sorted array: 1 3
Enter second sorted array: 2
Median is: 2.00000
>>>
================== RESTART: C:/Users/Hp/23MID061_LEETCODE_4.py ==================
Enter first sorted array: 1 2
Enter second sorted array: 3 4
Median is: 2.50000
>>>
```

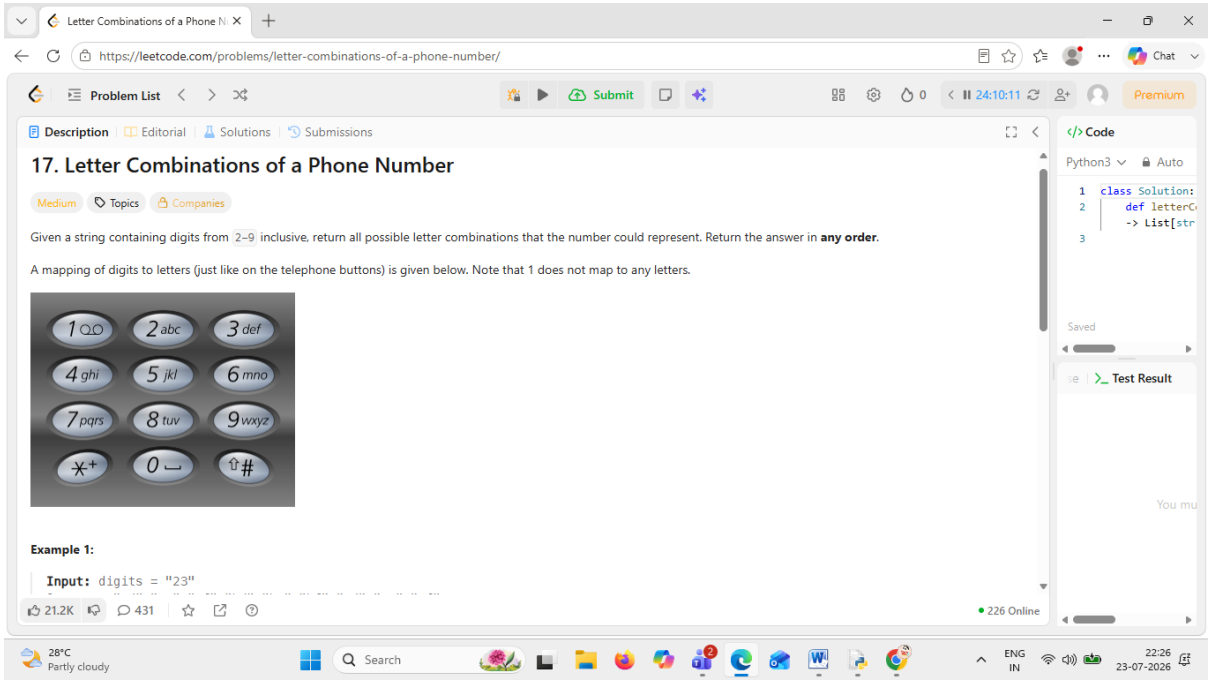
Justification for Data Structure Used :

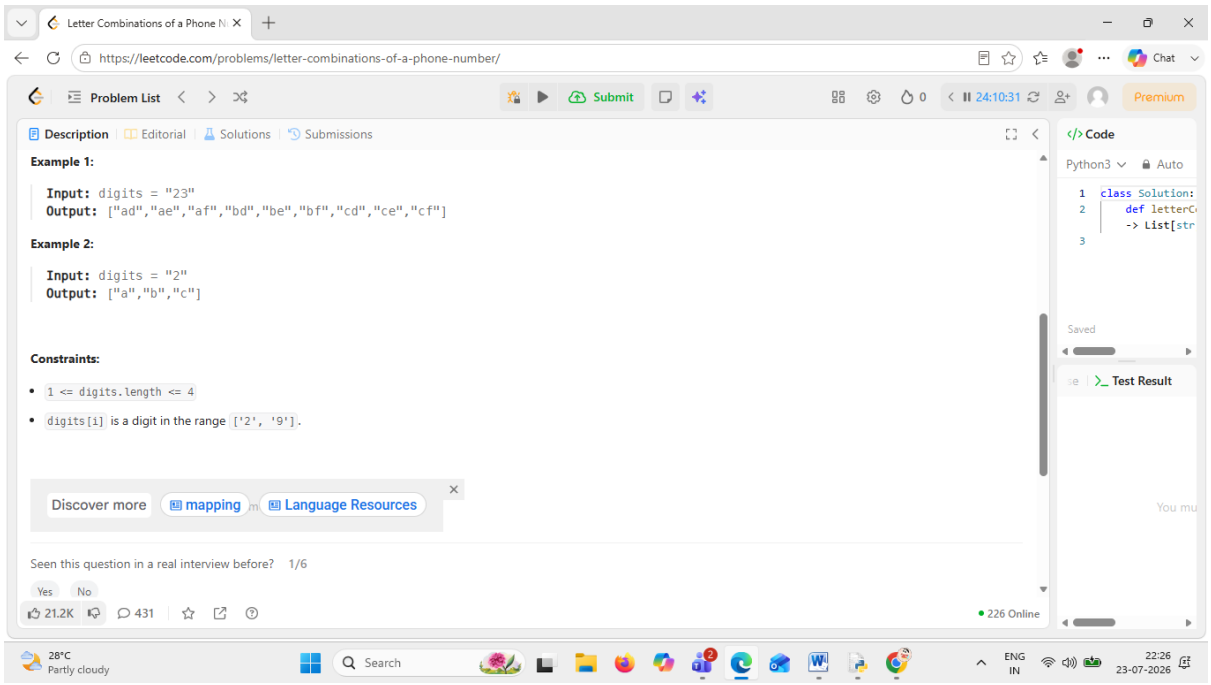
A **list** is used because it allows the elements of both input arrays to be stored together in a single merged array. This makes easy to access the middle elements and calculate the median.

Logic Used :

1. Read the two sorted arrays (nums1 and nums2) from the user.
2. Merge both arrays into a single list using the + operator.
3. Sort the merged list in ascending order using the sort() method.
4. Find the total number of elements in the merged list. If the number of elements is odd, the median is the middle element. If the number of elements is even, the median is the average of the two middle elements.

QUESTION -5 :

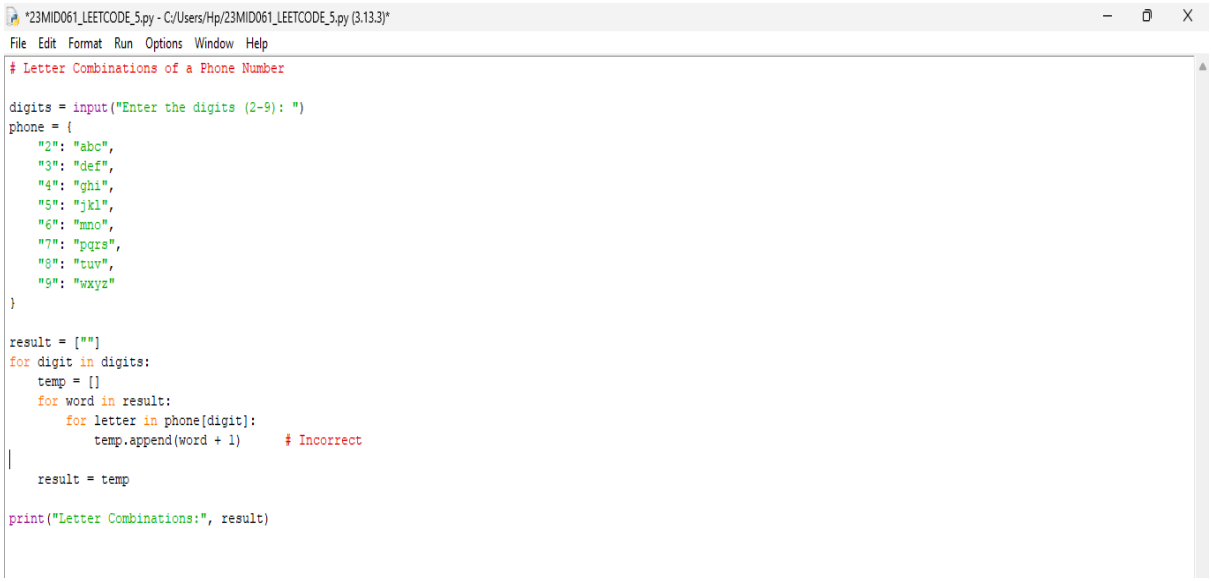




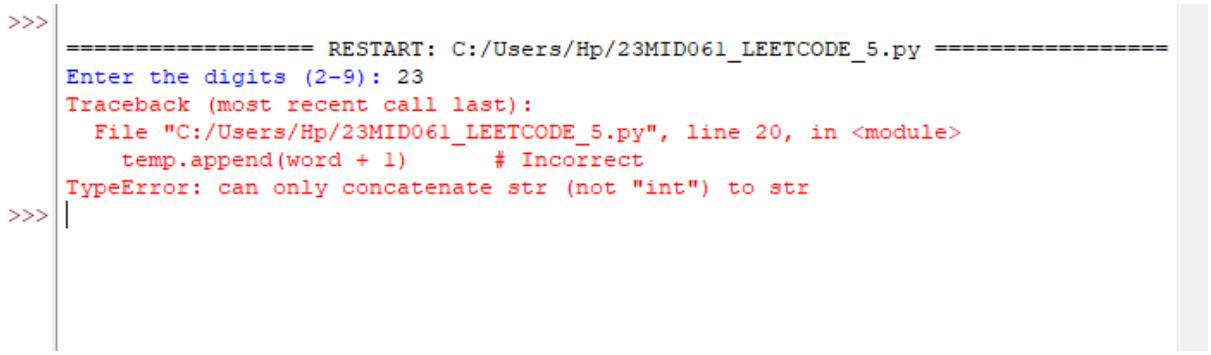
INPUT : A string containing digits from 2 to 9.

OUTPUT : A list containing all possible letter combinations for the given string as input.

CODE :



ERROR ENCOUNTERED :



Reason for the error in the output:

The variable word is a string, and it should be combined with another string to form a letter combination. By mistake, the integer 1 was used instead of letter. Since Python cannot combine a string with an integer, it raises a TypeError.

CORRECTED CODE :

23MID061_LEETCODE_5.py - C:/Users/Hp/23MID061_LEETCODE_5.py (3.13.3)

File Edit Format Run Options Window Help

```
# Letter Combinations of a Phone Number

#Input : A string containing digits from 2 to 9.

#Output: A list containing all possible letter combinations.

digits = input("Enter the digits (2-9): ")
phone = {
    "2": "abc",
    "3": "def",
    "4": "ghi",
    "5": "jkl",
    "6": "mno",
    "7": "pqrs",
    "8": "tuv",
    "9": "wxyz"
}

result = [""]

for digit in digits:
    temp = []
    for word in result:
        for letter in phone[digit]:
            temp.append(word + letter)

    result = temp

print("Letter Combinations:", result)
```

OUTPUT :

IDLE Shell 3.13.3

File Edit Shell Debug Options Window Help

```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================== RESTART: C:/Users/Hp/23MID061_LEETCODE_5.py ==================
Enter the digits (2-9): 23
Letter Combinations: ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce', 'cf']
>>>
================== RESTART: C:/Users/Hp/23MID061_LEETCODE_5.py ==================
Enter the digits (2-9): 2
Letter Combinations: ['a', 'b', 'c']
>>>
```

Justification for Data Structure Used :

- A **dictionary** is used to store the mapping between each digit and its corresponding letters on the telephone keypad.
- A **list** is used to store all the possible letter combinations generated from the given digits.

Logic Used :

1. Read the digit string from the user.
2. Store the digit-to-letter mapping in a dictionary.
3. Start with an empty string in a list.
4. For each digit, combine every existing string with each letter mapped to that digit and store the new combinations in a new list.
5. Display the final list of all possible letter combinations.

THANK YOU